จัดกลุ่มลูกค้าด้วย K-Means

ฝึกโมเดลด้วยข้อมูลชุดฝึก แล้วจัดกลุ่มลูกค้าใหม่ในข้อมูลชุดทดสอบ

ชื่อ-สกุล _____ รหัสนักศึกษา _____

รายวิชา _____ กลุ่มเรียน _____ วันที่ _____

1 จุดประสงค์

นักศึกษาต่อบล็อกใน ThaiPyBlocks เพื่อแบ่งลูกค้าเป็น **ชุดฝึก (train)** กับ **ชุดทดสอบ (test)** ให้ K-Means หากลุ่มจากชุดฝึก แล้วตรวจว่ากลุ่มที่ได้ใช้กับลูกค้าที่โมเดลไม่เคยเห็นได้ ทุกบล็อกคือคำสั่ง Python (pandas · scikit-learn) จริง ดูได้ในแท็บ "โค้ด Python"

เมื่อทำใบงานเสร็จ นักศึกษาจะ

- อธิบายได้ว่าทำไมต้องแยกชุดทดสอบไว้ก่อนฝึกโมเดล และใช้บล็อก `train_test_split` ได้
- ฝึก K-Means กับชุดฝึก แล้วจัดลูกค้าในชุดทดสอบเข้ากลุ่มที่เรียนได้
- เลือกจำนวนกลุ่ม k ด้วยวิธี elbow และวัดคุณภาพกลุ่มด้วยค่า silhouette (−1 ถึง 1 ยิ่งใกล้ 1 กลุ่มยิ่งแยกกันชัด)

2 สิ่งที่ต้องเตรียม

1. เปิดแอป ThaiPyBlocks จากลิงก์ที่ผู้สอนให้ บนคอมพิวเตอร์หรือแท็บเล็ต (จอกว้างต่อบล็อกได้สะดวกกว่า) ลิงก์ _____
2. ดาวน์โหลดไฟล์ `mall_customers.csv` จากผู้สอน
3. ในแอปกด **ข้อมูลของฉัน** → **เลือกไฟล์** (หรือลากไฟล์มาวาง) แอปแสดงจำนวนแถว คอลัมน์ และ 5 แถวแรกให้ตรวจว่าอ่านไฟล์ถูก
4. กดเมนู ⋯ → **ล้างพื้นที่ทำงาน** แล้วเริ่มต่อบล็อกใหม่ แตะบล็อกเพื่อดูคำสั่ง Python และความหมายภาษาไทยด้านล่าง
5. **ส่งงาน**: ใบงานนี้ที่กรอกตารางและตอบคำถามครบ พร้อมไฟล์ PDF จากปุ่ม **PDF** ในแท็บผลลัพธ์ของแอป (รันด้วยค่าตามใบงาน k = 5 ก่อนกด PDF)

ไฟล์ที่อัปโหลดอยู่ในเครื่องของนักศึกษาเท่านั้น ถ้ารีโหลดหน้าต้องอัปโหลดใหม่ (บล็อกที่ต่อไว้ไม่หาย)

3 ข้อมูล: ลูกค้าห้างสรรพสินค้า 200 คน

ไฟล์ `mall_customers.csv` เป็นข้อมูลจำลองเพื่อการสอน ลูกค้าแบ่งเป็น 5 กลุ่มตามรายได้และการใช้จ่าย แต่ไฟล์ไม่มีคอลัมน์บอกกลุ่ม โมเดลต้องหาเอง กิจกรรมใช้เฉพาะ income กับ spending ซึ่งมีช่วงค่าใกล้กัน จึงไม่ต้องปรับสเกลก่อนใช้ K-Means

คอลัมน์	ความหมาย	ค่าในไฟล์
`customer_id`	รหัสลูกค้า	C001–C200
`gender`	เพศ	หญิง 103 คน, ชาย 97 คน
`age`	อายุ (ปี)	18–70
`income`	รายได้ต่อเดือน (พันบาท)	16–108
`spending`	คะแนนการใช้จ่ายที่ห้างให้ ยิ่งมากยิ่งซื้อบ่อยและซื้อมาก	1–99

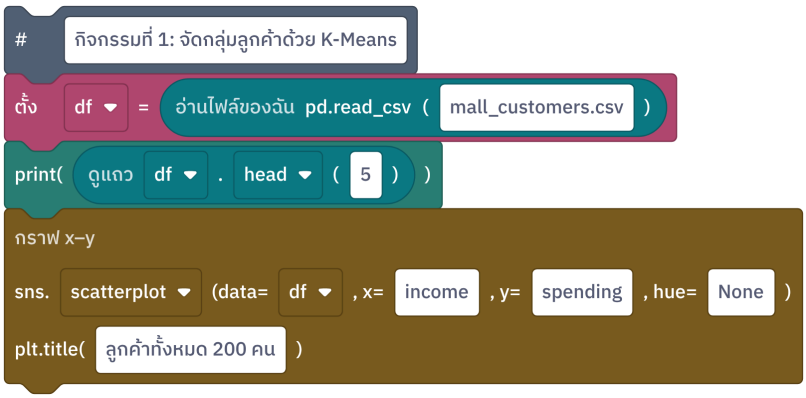
4 ขั้นตอนการต่อบล็อก

กด ▶ รัน ได้ทุกเมื่อ แอปรันใหม่ตั้งแต่บล็อกแรก หลังรันครั้งแรก ช่องชื่อคอลัมน์จะมีรายการให้เลือก เปลี่ยนชื่อตัวแปรได้ที่ลูกศร ▾ ของตัวแปร → เปลี่ยนชื่อตัวแปร...

■ **ข้อมูล › โหลด & สำรวจ** = หมวดบล็อกในแถบซ้าย (สีเดียวกับในแอป) **ตั้ง df = อ่านไฟล์ของฉัน** = บล็อกที่ลากมาต่อ

ตอนที่ 1 โหลดและดูข้อมูล

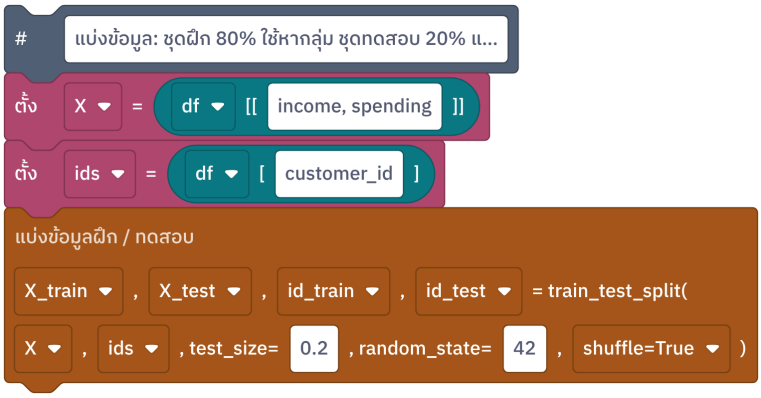
1. ■ **ข้อมูล › โหลด & สำรวจ** ลาก **ตั้ง df = อ่านไฟล์ของฉัน** มาวาง กดช่อง "เลือกไฟล์..." แล้วเลือก `mall_customers.csv`
2. ต่อ **print(ดูแถว df.head(5))** จากหมวดเดียวกัน
3. ■ **กราฟ** ต่อ **กราฟ x–y** เลือก scatterplot, data = df, x = income, y = spending, hue เว้นว่าง
4. กด ▶ รัน ต้องเห็นตาราง 5 แถวและกราฟจุด 200 จุด



บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

ตอนที่ 2 แบ่งชุดฝึก / ชุดทดสอบ

1. ■ **เตรียมข้อมูล** ลาก **ตั้ง X = ...** มาวาง เปลี่ยนค่าทางขวาเป็น **df[[ ]]** จาก ■ **ข้อมูล › เลือก & กรอง** แล้วเลือก income, spending
2. ■ **เตรียมข้อมูล** ลาก **ตั้ง y = df[ ]** เลือก customer_id แล้วเปลี่ยนชื่อตัวแปร y เป็น `ids`
3. ■ **เตรียมข้อมูล** ต่อ **แบ่งข้อมูลฝึก / ทดสอบ** ตั้งค่า X, ids, test_size = 0.2, random_state = 42 เลือก shuffle=True (ไม่ใช้ stratify เพราะไม่มีคลาส) และเปลี่ยนชื่อ y_train, y_test เป็น id_train, id_test
4. กด ▶ รัน ต้องเห็นข้อความ "train 160 แถว · test 40 แถว"



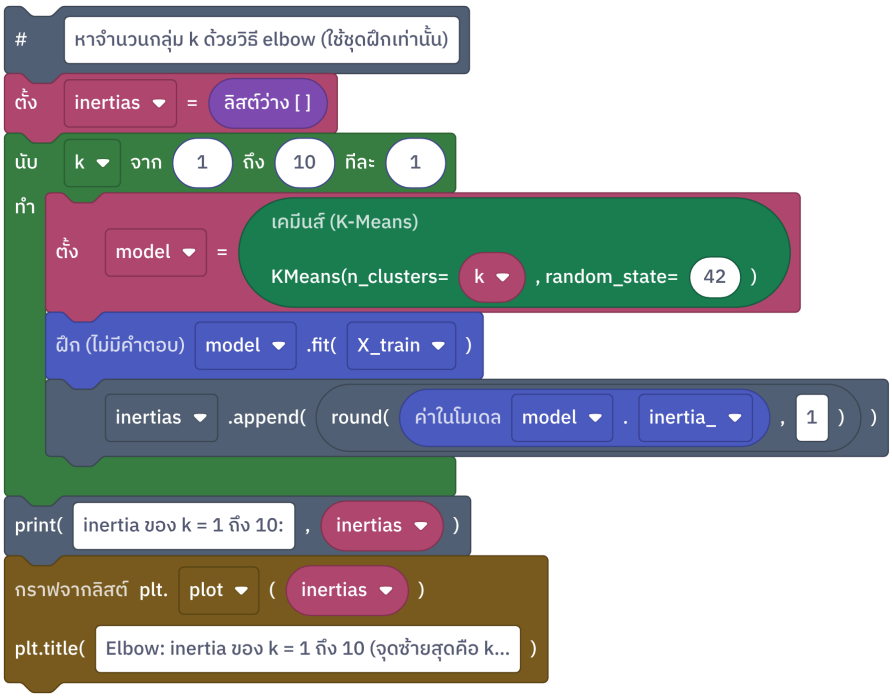
บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

โค้ด Python ที่ได้ (รหัสลูกค้าไม่ได้ใช้ฝึก แต่บอกว่าลูกค้าคนไหนอยู่ชุดใด)

```
X_train, X_test, id_train, id_test = train_test_split(X, ids, test_size=0.2,
random_state=42)
```

ตอนที่ 3 หาจำนวนกลุ่ม k ด้วยวิธี elbow (ใช้ชุดฝึกเท่านั้น)

1. ■ **Python › ตัวแปร** กด สร้างตัวแปร... ชื่อ `inertias` ลาก **ตั้ง inertias =** มาต่อ แล้วใส่ **ลิสต์ว่าง [ ]** จาก ■ **Python › ลิสต์**
2. ■ **Python › ลูป** ต่อ **นับ i จาก 1 ถึง 10 ทีละ 1** แล้วเปลี่ยนชื่อ i เป็น `k`
3. ในช่อง "ทำ" ของลูป ใส่ 3 บล็อกตามลำดับ
   - ■ **โมเดล › จัดกลุ่ม & ลดมิติ** **ตั้ง model = KMeans(...)** แล้วลากตัวแปร k มาวางทับเลข n_clusters
   - ■ **ฝึก & ทำนาย** **ฝึก (ไม่มีคำตอบ) model.fit( )** เลือก X_train
   - ■ **Python › แสดงผล** **.append( )** เลือก inertias ใส่ **round(, 1)** แล้วใส่ **ค่าในโมเดล model . inertia_** ไว้ใน round
4. ใต้ลูป ต่อ **print(ผลลัพธ์: , ...)** แก้ข้อความเป็น `inertia ของ k = 1 ถึง 10:` แล้วใส่ตัวแปร inertias
5. ■ **กราฟ** ต่อ **กราฟจากลิสต์** แบบ plot ใส่ inertias
6. กด ▶ รัน บันทึกค่าลงตารางที่ 1 จุดซ้ายสุดของกราฟคือ k = 1 แล้วหา "ข้อศอก" ที่เส้นเริ่มแบนราบ

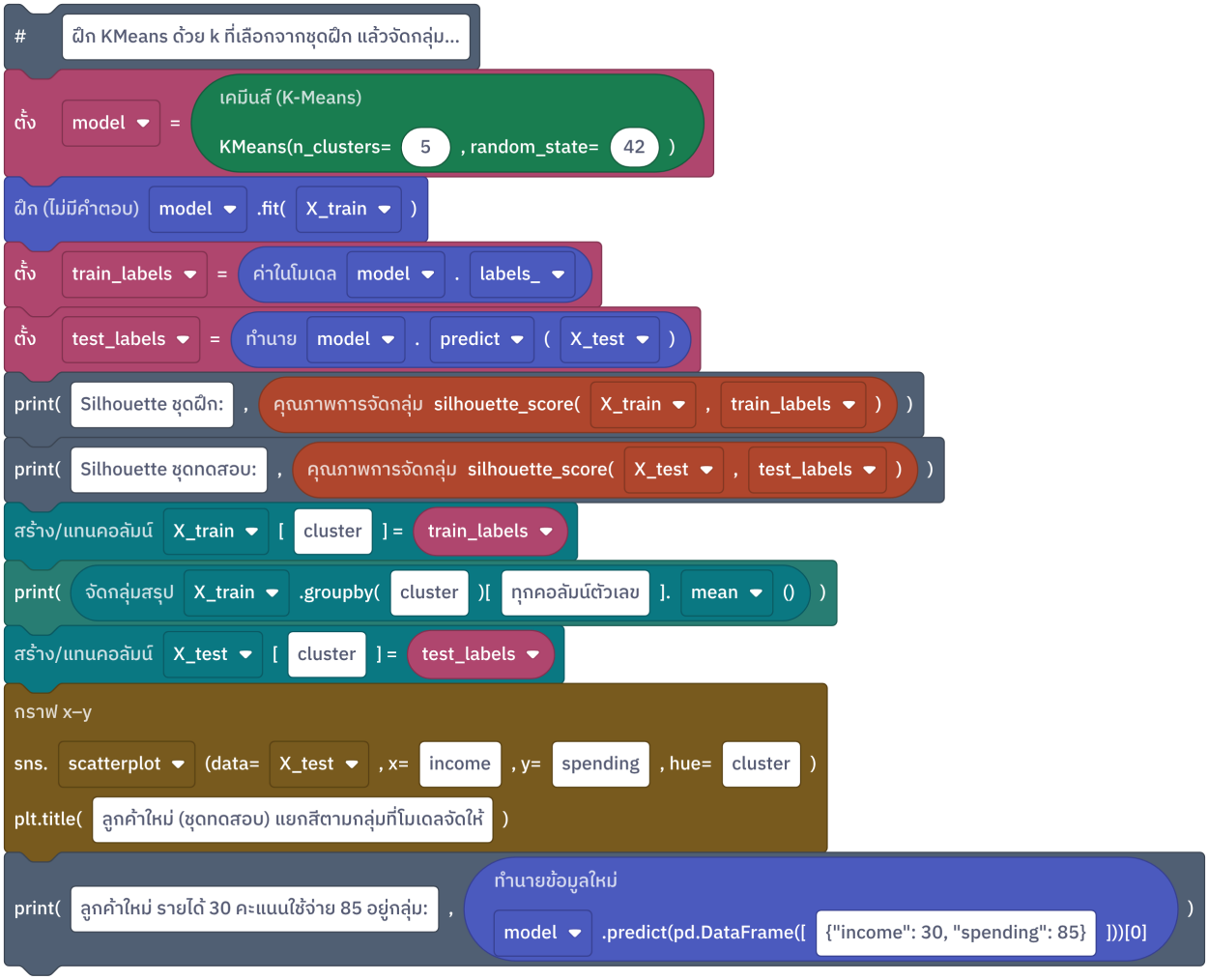


บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

ตอนที่ 4 ฝึกด้วยชุดฝึก แล้วจัดกลุ่มลูกค้าใหม่ในชุดทดสอบ

1. ต่อ **ตั้ง model = KMeans(...)** อีกครั้งใต้กราฟ (นอกลูป) ใส่ n_clusters = k ที่เลือกจากข้อศอก แล้วต่อ **ฝึก (ไม่มีคำตอบ) model.fit(X_train)**
2. สร้างตัวแปร `train_labels` แล้วต่อ **ตั้ง train_labels = ค่าในโมเดล model . labels_**
3. ■ **ฝึก & ทำนาย** ต่อ **ตั้ง y_pred = ทำนาย model.predict(X_test)** แล้วเปลี่ยนชื่อ y_pred เป็น `test_labels`
4. ■ **วัดผล** ใส่ **คุณภาพการจัดกลุ่ม silhouette_score(,)** ลงใน **print(,)** 2 บล็อก: ข้อความ `Silhouette ชุดฝึก:` กับ X_train, train_labels และ `Silhouette ชุดทดสอบ:` กับ X_test, test_labels
5. ■ **ข้อมูล › คำนวณคอลัมน์** **สร้าง/แทนคอลัมน์** เลือก X_train ชื่อคอลัมน์ cluster ใส่ train_labels แล้วต่อ **print(จัดกลุ่มสรุป X_train.groupby(cluster) ... mean())** จาก ■ **ข้อมูล › เลือก & กรอง** โดยเว้นช่องคอลัมน์ว่าง (= ทุกคอลัมน์ตัวเลข)
6. ต่อ **สร้าง/แทนคอลัมน์** อีกบล็อก เลือก X_test ชื่อ cluster ใส่ test_labels แล้วต่อ **กราฟ x–y** data = X_test, x = income, y = spending, hue = cluster
7. ■ **ฝึก & ทำนาย › ใช้งานโมเดล** ใส่ **ทำนายข้อมูลใหม่** ลงใน print แก้ค่าเป็น `{"income": 30, "spending": 85}`
8. กด ▶ รัน แล้วกดปุ่ม PDF เก็บผล

ลำดับในตอนที่ 4 สำคัญ ต้องทำนายและวัด silhouette ก่อนเพิ่มคอลัมน์ cluster ถ้าเพิ่มก่อน บล็อกทำนายจะแจ้งข้อผิดพลาด (ข้อมูลมี 3 คอลัมน์ แต่โมเดลฝึกมา 2 คอลัมน์) และ silhouette จะนับ cluster เป็นฟีเจอร์ด้วยจนค่าผิด



บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

5 ตารางบันทึกผล

บันทึกทศนิยม 3 ตำแหน่ง (เช่น 0.704) ทุกครั้งที่แก้ค่าในบล็อก ต้องกด ▶ รัน ใหม่ก่อนอ่านผล ทดลองเสร็จแล้วคืนค่าเดิมก่อนทำตารางถัดไป

ตารางที่ 1 inertia ของ k = 1–10

ตอนที่ 3: คัดจากบรรทัด "inertia ของ k = 1 ถึง 10" ตามลำดับ แล้วหาว่าแต่ละ k ลดลงจากค่าก่อนหน้าเท่าไร

k	inertia	ลดลงจาก k ก่อนหน้า
1		–
2		
3		
4		
5		
6		
7		
8		
9		
10		

ข้อศอกของกราฟอยู่ที่ k = _____

ตารางที่ 2 จำนวนกลุ่มกับคุณภาพการจัดกลุ่ม

แก้ n_clusters ในบล็อก KMeans ที่อยู่นอกลูปเป็น 3, 4, 5, 6 ทีละค่า แล้วรัน

k	Silhouette ชุดฝึก	Silhouette ชุดทดสอบ
3		
4		
5		
6		

ตารางที่ 3 ตั้งชื่อกลุ่มลูกค้า (k = 5)

คัดค่าเฉลี่ยจากตาราง groupby แล้วตั้งชื่อกลุ่ม เช่น "รายได้สูง ใช้จ่ายน้อย"

cluster	income เฉลี่ย (พันบาท)	spending เฉลี่ย	ชื่อกลุ่ม
0			
1			
2			
3			
4			

ตารางที่ 4 เปลี่ยนวิธีแบ่งข้อมูล (k = 5)

แก้ test_size หรือ random_state ในบล็อกแบ่งข้อมูลตามแถว แล้วรัน

test_size	random_state	แถว ฝึก / ทดสอบ	Silhouette ชุดฝึก	Silhouette ชุดทดสอบ	ลูกค้าใหม่ (30, 85) อยู่กลุ่ม
0.2	42				
0.2	7				
0.5	42				

6 คำถามท้ายใบงาน

ตอบสั้น ๆ พร้อมอ้างตัวเลขจากตาราง

1 กราฟจุดในตอนที่ 1 เห็นลูกค้ากี่กลุ่ม ตรงกับ k ที่ข้อศอกของตารางที่ 1 หรือไม่

2 ทำไมต้องหา k และฝึก KMeans ด้วยชุดฝึกเท่านั้น ถ้าใช้ลูกค้าทั้ง 200 คนฝึก จะรู้ได้อย่างไรว่ากลุ่มที่ได้ใช้กับลูกค้าใหม่ได้

3 ตารางที่ 2: ถ้าเลือก k จาก Silhouette ชุดทดสอบ จะได้ k เท่าไร ต่างจากชุดฝึกหรือไม่ ควรใช้ชุดไหนเลือก k เพราะอะไร

4 ตารางที่ 4: เลขกลุ่มของลูกค้าใหม่เปลี่ยนไปตามการแบ่งข้อมูล แปลว่าเขาย้ายกลุ่มจริงหรือไม่ เขาเป็นลูกค้ากลุ่มไหนในตารางที่ 3 และห้างควรเสนออะไรให้เขา

ทำนายการงอกของเมล็ดด้วย Neural Network

ฝึกโมเดลด้วยข้อมูลชุดฝึก แล้ววัดความแม่นยำกับข้อมูลชุดทดสอบ

ชื่อ-สกุล _____ รหัสนักศึกษา _____

รายวิชา _____ กลุ่มเรียน _____ วันที่ _____

1 จุดประสงค์

นักศึกษาต่อบล็อกใน ThaiPyBlocks เพื่อแบ่งข้อมูลเป็น **ชุดฝึก (train)** กับ **ชุดทดสอบ (test)** ฝึก Neural Network (`MLPClassifier`) ด้วยชุดฝึก แล้ววัด accuracy กับชุดทดสอบที่โมเดลไม่เคยเห็น เทียบกับ `LogisticRegression` ซึ่งแบ่งข้อมูลได้ด้วยเส้นตรงเท่านั้น

เมื่อทำใบงานเสร็จ นักศึกษาจะ

- แยกฟีเจอร์ X กับคำตอบ y แล้วแบ่งชุดฝึก / ชุดทดสอบโดยรักษาสัดส่วนคลาส (stratify)
- ปรับสเกลด้วยค่าที่เรียนจากชุดฝึกเท่านั้น และบอกได้ว่าทำไม
- อ่าน accuracy ของชุดฝึกเทียบกับชุดทดสอบ เพื่อดูว่าโมเดลเรียนน้อยไป (underfitting) หรือจำข้อมูลฝึก (overfitting)
- บอกได้ว่าทำไม Neural Network แยกข้อมูลที่เส้นแบ่งเป็นเส้นโค้งได้ แต่โมเดลเส้นตรงทำไม่ได้

2 สิ่งที่ต้องเตรียม

1. เปิดแอป ThaiPyBlocks จากลิงก์ที่ผู้สอนให้ บนคอมพิวเตอร์หรือแท็บเล็ต (จอกว้างต่อบล็อกได้สะดวกกว่า) ลิงก์ _____
2. ดาวน์โหลดไฟล์ `seed_germination.csv` จากผู้สอน
3. ในแอปกด **ข้อมูลของฉัน** → **เลือกไฟล์** (หรือลากไฟล์มาวาง) แอปแสดงจำนวนแถว คอลัมน์ และ 5 แถวแรกให้ตรวจว่าอ่านไฟล์ถูก
4. กดเมนู ⋯ → **ล้างพื้นที่ทำงาน** แล้วเริ่มต่อบล็อกใหม่ แตะบล็อกเพื่อดูคำสั่ง Python และความหมายภาษาไทยด้านล่าง
5. **ส่งงาน**: ใบงานนี้ที่กรอกตารางและตอบคำถามครบ พร้อมไฟล์ PDF จากปุ่ม **PDF** ในแท็บผลลัพธ์ของแอป (รันด้วยค่าตามใบงาน MLP (16,) ก่อนกด PDF)

ไฟล์ที่อัปโหลดอยู่ในเครื่องของนักศึกษาเท่านั้น ถ้ารีโหลดหน้าต้องอัปโหลดใหม่ (บล็อกที่ต่อไว้ไม่หาย)

3 ข้อมูล: ผลการเพาะเมล็ด 400 ถาด

ไฟล์ `seed_germination.csv` เป็นข้อมูลจำลองเพื่อการสอน เมล็ดงอกดีเมื่ออุณหภูมิและความชื้น "พอดี" ทั้งคู่ ส่วนใหญ่อยู่ที่ราว 22–33 °C และความชื้น 44–77% จุด "งอก" จึงรวมกันเป็นวงรีกลางกราฟ เส้นตรงเส้นเดียวแบ่งไม่ได้

คอลัมน์	ความหมาย	ค่าในไฟล์
`temperature`	อุณหภูมิเฉลี่ยระหว่างเพาะ (°C)	10.2–40.0
`humidity`	ความชื้นในดิน (%)	10–95
`germinated`	ผลการเพาะ = คำตอบ y	งอก, ไม่งอก

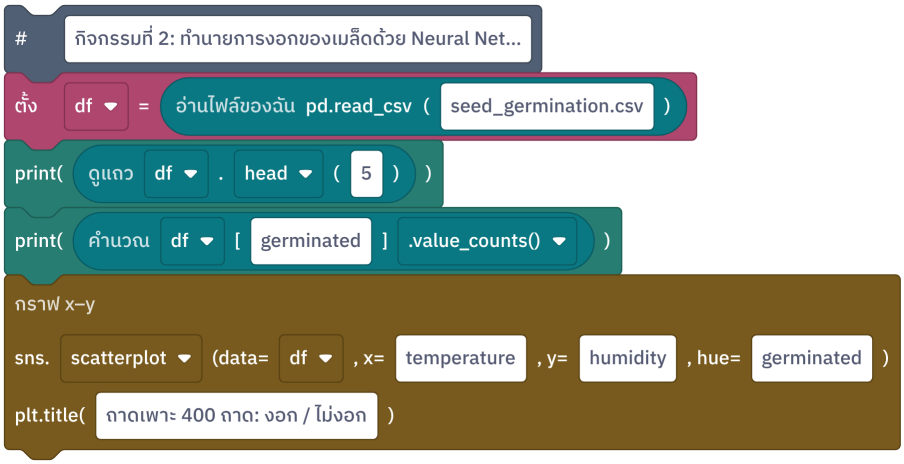
4 ขั้นตอนการต่อบล็อก

กด ▶ รัน ได้ทุกเมื่อ แอปรันใหม่ตั้งแต่บล็อกแรก หลังรันครั้งแรก ช่องชื่อคอลัมน์จะมีรายการให้เลือก

ข้อมูล › โหลด & สำรวจ = หมวดบล็อกในแถบซ้าย (สีเดียวกับในแอป) `ตั้ง df = อ่านไฟล์ของฉัน` = บล็อกที่ลากมาต่อ

ตอนที่ 1 โหลดและดูข้อมูล

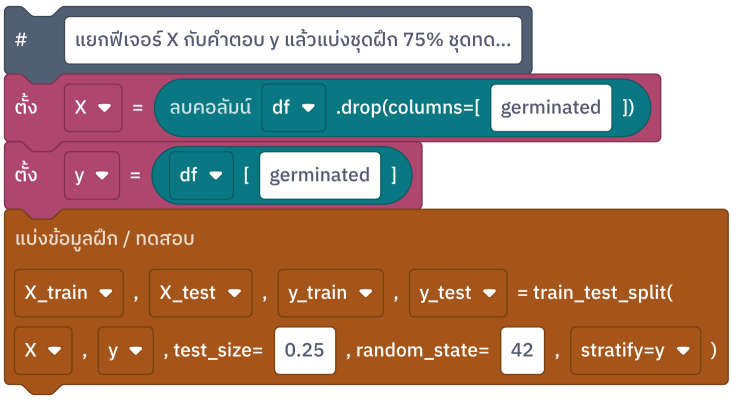
1. **ข้อมูล › โหลด & สำรวจ** ลาก `ตั้ง df = อ่านไฟล์ของฉัน` มาวาง เลือก `seed_germination.csv` แล้วต่อ `print( ดูแถว df.head(5) )`
2. **ทำความสะอาด & ปรับสเกล › ตรวจคุณภาพข้อมูล** ต่อ `print( df[ ].value_counts() )` เลือก germinated เพื่อนับถาดงอก / ไม่งอก
3. **กราฟ** ต่อ `กราฟ x–y` scatterplot, data = df, x = temperature, y = humidity, hue = germinated
4. กด ▶ รัน แล้วดูว่าจุด "งอก" อยู่ตรงไหนของกราฟ



บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

ตอนที่ 2 แยก X, y และแบ่งชุดฝึก / ชุดทดสอบ

1. **เตรียมข้อมูล** ลาก `ตั้ง X = ลบคอลัมน์ df.drop(columns=[ ])` เลือก germinated (เหลือ temperature, humidity)
2. **เตรียมข้อมูล** ลาก `ตั้ง y = df[ ]` เลือก germinated
3. **เตรียมข้อมูล** ต่อ `แบ่งข้อมูลฝึก / ทดสอบ` test_size = 0.25, random_state = 42 เลือก stratify=y เพื่อให้สัดส่วนงอก / ไม่งอกเท่ากันทั้งสองชุด
4. กด ▶ รัน ต้องเห็นข้อความ "train 300 แถว · test 100 แถว"

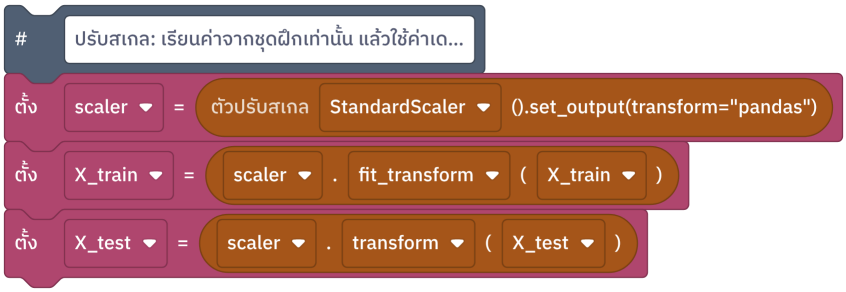


บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

ตอนที่ 3 ปรับสเกลด้วยค่าจากชุดฝึกเท่านั้น

1. ■ **เตรียมข้อมูล** ต่อ 3 บล็อกเรียงกัน: ตั้ง scaler = StandardScaler() → ตั้ง X_train = scaler.fit_transform(X_train) → ตั้ง X_test = scaler.transform(X_test)

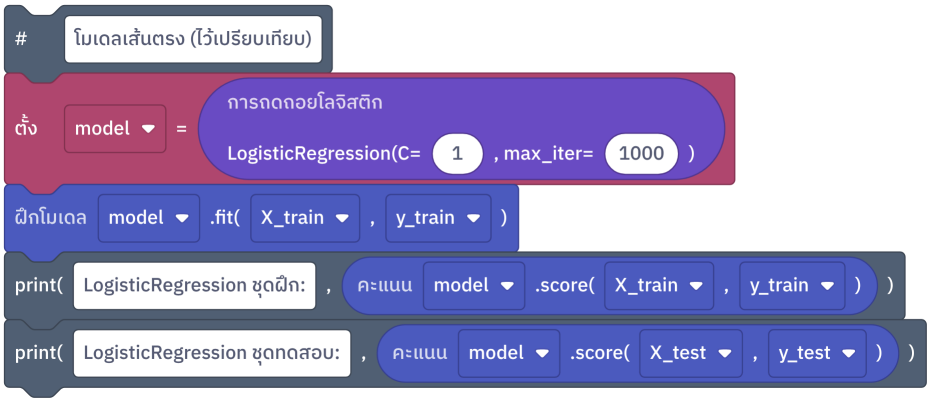
> fit_transform เรียนค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานจากชุดฝึกแล้วแปลง ส่วน transform ใช้ค่าเดิมนั้นแปลงชุดทดสอบ ห้าม fit กับชุดทดสอบ เพราะเท่ากับให้โมเดลแอบดูข้อสอบ



บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

ตอนที่ 4 โมเดลเส้นตรงไว้เปรียบเทียบ

1. ■ **โมเดล › จำแนกประเภท** ต่อ ตั้ง model = LogisticRegression(C=1, max_iter=1000)
2. ■ **ฝึก & ทำนาย** ต่อ ฝึกโมเดล model.fit(X_train, y_train)
3. ใส่ คะแนน model.score(,) ลงใน print(,) 2 บล็อก: `LogisticRegression ชุดฝึก:` กับ X_train, y_train และ `LogisticRegression ชุดทดสอบ:` กับ X_test, y_test

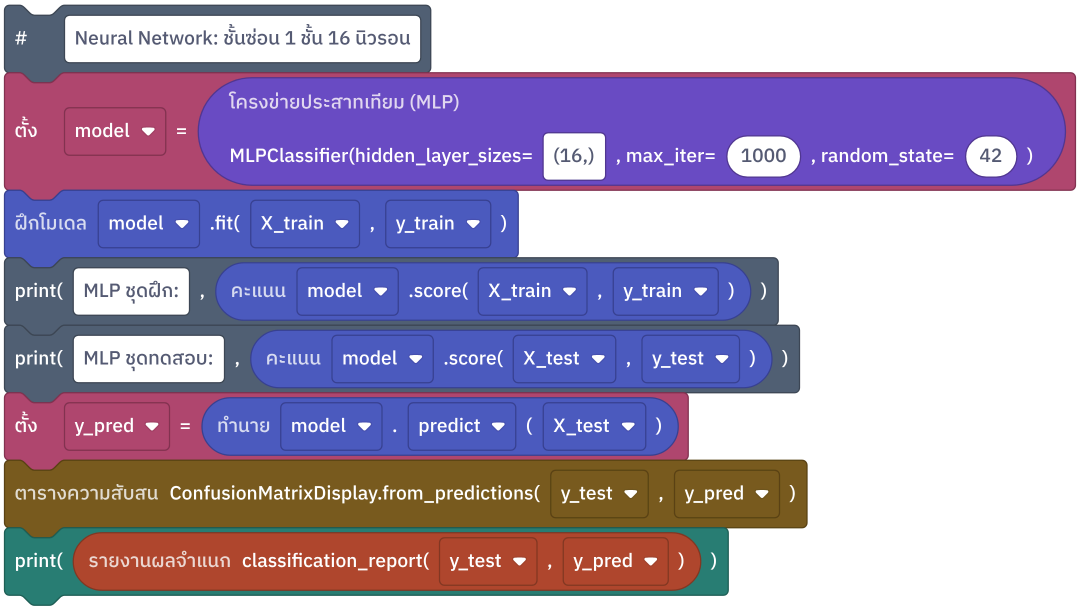


บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

ตอนที่ 5 Neural Network

1. ■ **โมเดล › จำแนกประเภท** ต่อ ตั้ง model = MLPClassifier(...) แก้ hidden_layer_sizes เป็น `(16,)` (ชั้นซ่อน 1 ชั้น 16 นิวรอน) และ max_iter เป็น 1000
2. ต่อ ฝึกโมเดล model.fit(X_train, y_train) และ print คะแนน 2 บล็อกแบบตอนที่ 4 ใช้ข้อความ `MLP ชุดฝึก:` กับ `MLP ชุดทดสอบ:` (คลิกขวาที่บล็อก → ทำซ้ำบล็อก แล้วแก้ข้อความได้)
3. ■ **ฝึก & ทำนาย** ต่อ ตั้ง y_pred = ทำนาย model.predict(X_test)
4. ■ **กราฟ** ต่อ ตารางความสับสน (y_test, y_pred) และ ■ **วัดผล** ต่อ print(รายงานผลจำแนก classification_report(y_test, y_pred))
5. กด ▶ รัน (MLP ใช้เวลาฝึกไม่กี่วินาที) แล้วกดปุ่ม PDF เก็บผล

> ตารางความสับสน: แถว = คำตอบจริง คอลัมน์ = สิ่งที่โมเดลทาย ช่องแนวทแยงคือทายถูก



บล็อกของตอนนี้เมื่อต่อเสร็จ ใช้ตรวจลำดับของตนเอง

5 ตารางบันทึกผล

บันทึกทศนิยม 3 ตำแหน่ง ทุกครั้งที่แก้ค่าในบล็อก ต้องกด ▶ รัน ใหม่ก่อนอ่านผล ทดลองเสร็จแล้วคืนค่าเดิม

ตารางที่ 1 จำนวนถาดงอก / ไม่งอก

แถวแรกจาก value_counts · แถวที่สองจากคอลัมน์ support ของ classification_report

	งอก	ไม่งอก	รวม	สัดส่วนงอก
ทั้งไฟล์			400	
ชุดทดสอบ			100	

ตารางที่ 2 เปรียบเทียบโมเดลทำนายการงอก

แก้ค่าในบล็อก MLPClassifier ทีละแถว แล้วรัน (แถว LogisticRegression ได้จากทุกการรัน) แถวสุดท้าย: คลิกขวาที่ 3 บล็อกปรับสเกล → ปิดการทำงานบล็อก

โมเดล	hidden_layer_sizes	max_iter	Accuracy ชุดฝึก	Accuracy ชุดทดสอบ	ฝึก − ทดสอบ	มีคำเตือนไหม
LogisticRegression	–	1000				
MLP	(2,)	1000				
MLP	(8,)	1000				
MLP	(16,)	1000				
MLP	(100,)	1000				
MLP	(16, 16)	1000				
MLP	(16,)	300				
MLP ไม่ปรับสเกล	(16,)	1000				

6 คำถามท้ายใบงาน

ตอบสั้น ๆ พร้อมอ้างตัวเลขจากตาราง

1 ถ้าทายว่า "ไม่งอก" ทุกถาด จะได้ accuracy บนชุดทดสอบเท่าไร (ใช้ตารางที่ 1) LogisticRegression ทำได้ดีกว่านี้หรือไม่ เพราะอะไร (ดูรูปร่างของจุดในกราฟ)

2 MLP (2,) ได้ accuracy เท่ากับคำตอบข้อ 1 พอดี ดูแถว งอก ใน classification_report แล้วอธิบายว่าโมเดลทำอะไรอยู่

3 ตารางที่ 2: แบบไหนได้ accuracy ชุดทดสอบสูงสุด เป็นแบบเดียวกับที่ชุดฝึกสูงสุดหรือไม่ ค่า ฝึก – ทดสอบ ที่กว้างขึ้นบอกอะไร

4 คำเตือนที่ max_iter = 300 หมายความว่าอะไร แก้อย่างไร

5 ทำไมต้อง fit ตัวปรับสเกลกับชุดฝึก แล้วแค่ transform ชุดทดสอบ แถว "ไม่ปรับสเกล" บอกอะไรเกี่ยวกับ Neural Network

6 ถ้าจะทำนายถาดใหม่ 27 °C ความชื้น 60% ด้วยโมเดลนี้ ต้องทำอะไรกับค่าก่อนส่งให้โมเดล **ข้อท้าทาย**

เฉลยใบงานที่ 1 และ 2 สำหรับผู้สอน

ผลที่ควรได้ ตารางบันทึกผล และแนวคำตอบ (ไม่แจกนักศึกษา)

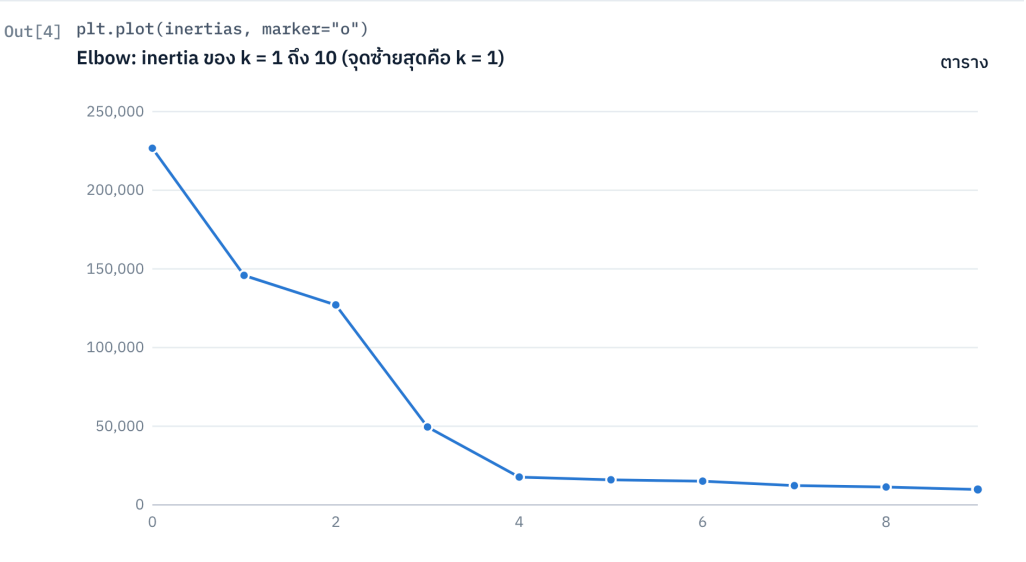
ตัวเลขทุกค่าได้จากการรันเฉลยใน ThaiPyBlocks และตรงกับโค้ด Python ที่แอปสร้างเมื่อรันด้วย pandas 3.0 และ scikit-learn 1.8 (รุ่นอื่นอาจต่างที่ทศนิยมตำแหน่งท้าย ๆ แต่ข้อสรุปเหมือนเดิม) เปิดเฉลยบล็อกได้ที่เมนู ⋯ → เปิดโครงงาน → `solution_kmeans.json` / `solution_neural_network.json` แล้วอัปโหลด CSV และกด ▶ รัน

1 ใบงานที่ 1 K-Means (test_size 0.2, random_state 42)

ตารางที่ 1 inertia ของ k = 1–10

k	inertia	ลดลงจาก k ก่อนหน้า
1	226,726.5	–
2	145,867.4	80,859.1
3	127,108.7	18,758.7
4	49,493.0	77,615.7
5	17,688.4	31,804.6
6	15,921.9	1,766.5
7	15,089.3	832.6
8	12,249.1	2,840.2
9	11,332.2	916.9
10	9,802.4	1,529.8

ข้อศอกอยู่ที่ **k = 5**: จาก k = 4 ลดลง 31,804.6 แต่ถัดไปลดลงแค่ 1,766.5



กราฟ elbow ที่ควรได้ (จุดที่ 4 นับจาก 0 คือ k = 5)

ตารางที่ 2 จำนวนกลุ่มกับคุณภาพการจัดกลุ่ม

k	Silhouette ชุดฝึก	Silhouette ชุดทดสอบ
3	0.399	0.509
4	0.593	0.677
5	**0.704**	0.651
6	0.626	0.557

k = 5 ดีที่สุดบนชุดฝึก แต่ชุดทดสอบให้ k = 4 สูงกว่า เพราะชุดทดสอบมีลูกค้ากลุ่มรายได้น้อย ใช้จ่ายสูงเพียง 2 คน (k = 4 รวมกลุ่มนี้เข้ากับกลุ่มกลาง) ค่าจากลูกค้า 40 คนจึงแกว่งง่าย ใช้ประกอบคำถามข้อ 3

ตารางที่ 3 ตั้งชื่อกลุ่มลูกค้า (k = 5)

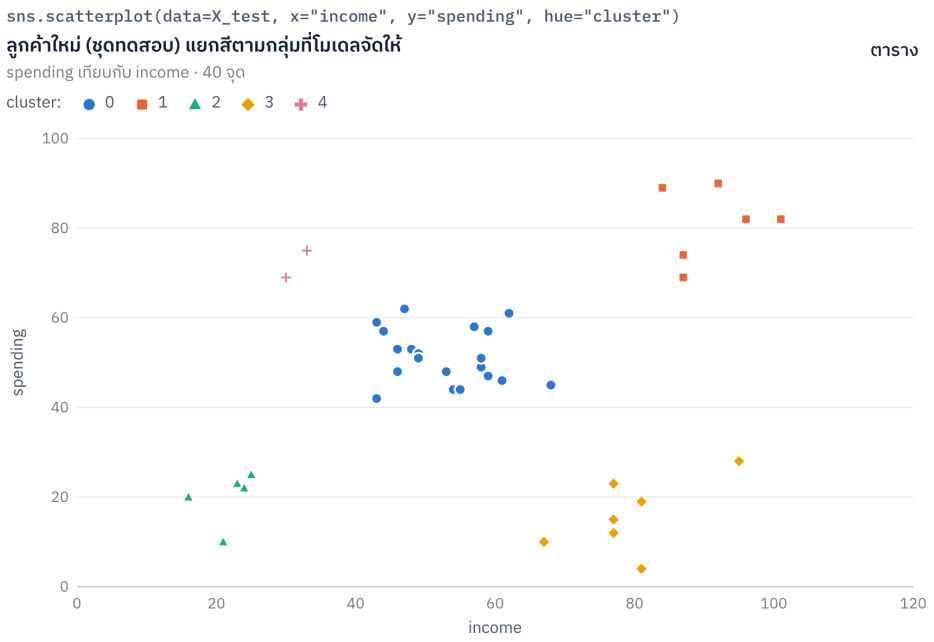
cluster	income เฉลี่ย (พันบาท)	spending เฉลี่ย	ชื่อกลุ่ม (ตัวอย่าง)
0	55.2	50.2	รายได้กลาง ใช้จ่ายกลาง
1	87.9	82.9	รายได้สูง ใช้จ่ายสูง
2	23.1	19.3	รายได้น้อย ใช้จ่ายน้อย
3	88.3	16.2	รายได้สูง ใช้จ่ายน้อย
4	24.2	84.2	รายได้น้อย ใช้จ่ายสูง

ตารางที่ 4 เปลี่ยนวิธีแบ่งข้อมูล (k = 5)

test_size	random_state	แถว ฝึก / ทดสอบ	Silhouette ชุดฝึก	Silhouette ชุดทดสอบ	ลูกค้าใหม่ (30, 85)
0.2	42	160 / 40	0.704	0.651	กลุ่ม 4
0.2	7	160 / 40	0.706	0.658	กลุ่ม 2
0.5	42	100 / 100	0.712	0.682	กลุ่ม 4

ทั้งสามแถวคือกลุ่ม "รายได้น้อย ใช้จ่ายสูง" เลขกลุ่มเป็นแค่ป้ายที่ K-Means ตั้งใหม่ทุกครั้งที่ฝึก

Out[8]
```
sns.scatterplot(data=X_test, x="income", y="spending", hue="cluster")
```



ลูกค้าในชุดทดสอบ 40 คน แยกสีตามกลุ่มที่โมเดล k = 5 จัดให้

แนวคำตอบใบงานที่ 1

1. เห็น 5 กลุ่ม (สี่มุมกับตรงกลาง) ตรงกับข้อศอก k = 5
2. ชุดทดสอบคือลูกค้าที่โมเดลไม่เคยเห็น Silhouette ชุดทดสอบ 0.651 ใกล้ชุดฝึก 0.704 จึงบอกได้ว่ากลุ่มใช้กับลูกค้าใหม่ได้ ถ้าใช้ทั้ง 200 คนฝึกจะไม่มีข้อมูลเหลือไว้ตรวจ
3. ชุดทดสอบให้ k = 4 แต่ต้องเลือก k จากชุดฝึก (k = 5) การเลือกด้วยชุดทดสอบคือเอาข้อสอบมาใช้ตอนฝึก คะแนนทดสอบจะดีเกินจริง และลูกค้า 40 คนน้อยเกินกว่าจะใช้ตัดสินเอง
4. ไม่ได้ย้าย เลขกลุ่มเปลี่ยนเพราะฝึกใหม่ด้วยชุดฝึกคนละชุด ค่าเฉลี่ยของกลุ่มบอกว่าเขาอยู่กลุ่ม "รายได้น้อย ใช้จ่ายสูง" ทุกครั้ง เหมาะกับโปรโมชันผ่อน 0% หรือคูปองส่วนลด

2 ใบงานที่ 2 Neural Network (test_size 0.25 แบบ stratify, random_state 42)

ตารางที่ 1 จำนวนถาดงอก / ไม่งอก

	งอก	ไม่งอก	รวม	สัดส่วนงอก
ทั้งไฟล์	164	236	400	0.41
ชุดทดสอบ	41	59	100	0.41

stratify ทำให้สัดส่วนงอกเท่ากันทั้งสองชุด การทาย "ไม่งอก" ทุกถาดได้ accuracy 0.59 บนชุดทดสอบ

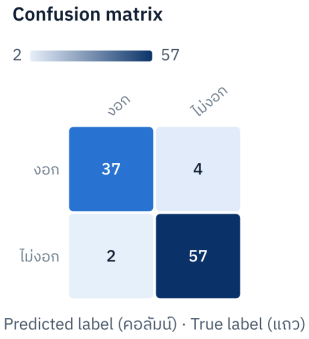
ตารางที่ 2 เปรียบเทียบโมเดล

โมเดล	hidden_layer_sizes	max_iter	ชุดฝึก	ชุดทดสอบ	ฝึก − ทดสอบ	คำเตือน
LogisticRegression	–	1000	0.450	0.460	−0.010	ไม่มี
MLP	(2,)	1000	0.590	0.590	0.000	ไม่มี*
MLP	(8,)	1000	0.947	0.910	0.037	ConvergenceWarning
MLP	**(16,)**	1000	0.963	**0.940**	0.023	ไม่มี
MLP	(100,)	1000	0.970	0.920	0.050	ไม่มี
MLP	(16, 16)	1000	0.977	0.920	0.057	ไม่มี
MLP	(16,)	300	0.913	0.880	0.033	ConvergenceWarning
MLP ไม่ปรับสเกล	(16,)	1000	0.747	0.780	−0.033	ไม่มี

* ใน Colab จะมี UndefinedMetricWarning จาก classification_report เพราะไม่มีถาดใดถูกทายว่างอก

ตารางความสับสนของ MLP (16,): งอก → งอก 37, งอก → ไม่งอก 4, ไม่งอก → งอก 2, ไม่งอก → ไม่งอก 57 · แถว งอก ใน classification_report: precision 0.95 recall 0.90 f1 0.93

```
Out[8] ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap="Blues")
```



ตารางความสับสนที่ควรได้ (MLP 16 นิวรอน)

แนวคำตอบใบงานที่ 2

1. ได้ 0.59 LogisticRegression ได้ 0.46 แย่กว่าการทายแบบนั้น เพราะถาดงอกรวมเป็นวงรีกลางกราฟและถูกล้อมด้วยถาดไม่งอก เส้นตรงเส้นเดียวแบ่งไม่ได้
2. โมเดลทาย "ไม่งอก" ทุกถาด แถว งอก มี precision และ recall เป็น 0.00 นิวรอน 2 ตัวน้อยเกินจะสร้างเส้นแบ่งโค้ง (underfitting)
3. (16,) ได้ชุดทดสอบสูงสุด 0.94 ส่วน (16, 16) ได้ชุดฝึกสูงสุด 0.977 แต่ชุดทดสอบเหลือ 0.92 ส่วนต่างที่กว้างขึ้น (0.023 → 0.057) คือสัญญาณว่าโมเดลเริ่มจำข้อมูลฝึก (overfitting)
4. ฝึกครบ 300 รอบแล้วค่า loss ยังลดไม่สุด (ยังไม่ลู่เข้า) accuracy จึงต่ำกว่า (0.913 / 0.880) แก้โดยเพิ่ม max_iter เป็น 1000
5. ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานต้องมาจากชุดฝึกเท่านั้น ไม่เช่นนั้นข้อมูลทดสอบรั่วเข้าการฝึก เมื่อไม่ปรับสเกล accuracy ตกเหลือ 0.747 / 0.780 เพราะ Neural Network ไวต่อขนาดตัวเลขของแต่ละคอลัมน์
6. แปลงค่าใหม่ด้วย scaler.transform ตัวเดิมก่อนทำนาย หรือใช้บล็อก "ไปป์ไลน์" รวม StandardScaler กับ MLPClassifier ให้ปรับสเกลอัตโนมัติ